

# Data Analytics Internship — Technical Report

Task 1: EDA & Cleaning · Task 2: Customer Segmentation · Task 3: Hypothesis Testing & Forecasting

*Dataset: Online Retail (UCI Machine Learning Repository)*

*541,909 transactions · UK-based online retailer · Dec 2010 – Dec 2011*

*This single document contains the full narrative report followed by the complete, executable source code for all three tasks.*

## PART A — NARRATIVE REPORT

### Task 1 — Exploratory Data Analysis & Integrity Cleaning Pipeline

Objective: Load a large raw industry dataset into Python/Pandas, diagnose data abnormalities, execute outlier filtering treatments, impute missing parameters systematically, and plot descriptive variable distributions to identify key patterns.

#### 1. Data Overview

The raw dataset contains 541,909 transaction-level rows across 8 columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country.

#### 2. Diagnosed Data Abnormalities

- Cancelled orders — InvoiceNo values beginning with 'C' (reversed transactions, not sales).
- Non-positive Quantity and UnitPrice values — data entry errors or adjustment rows.
- Missing CustomerID (~25% of rows) — unlinked / guest-checkout transactions.
- Missing Description values for a subset of StockCodes.
- Exact duplicate rows from repeated logging.

#### 3. Cleaning & Imputation Treatments

- Cancelled transactions separated into a dedicated returns frame; excluded from sales analysis.
- Rows with non-positive Quantity/UnitPrice (post-cancellation-removal) dropped as invalid records.
- Missing Description values imputed via a systematic StockCode → most-frequent-Description lookup.
- Missing CustomerID retained for revenue-level EDA, excluded from customer-level segmentation (Task 3), since no valid imputation exists for a customer identity.
- Duplicate rows dropped.

#### 4. Outlier Filtering (IQR Method)

A  $1.5 \times \text{IQR}$  filter was applied to Quantity and UnitPrice to remove extreme bulk-order and pricing outliers, with sensible floors (Quantity  $\geq 1$ , UnitPrice  $\geq \text{£}0.01$ ) preserved so that legitimate low-value retail transactions were not misclassified as outliers.

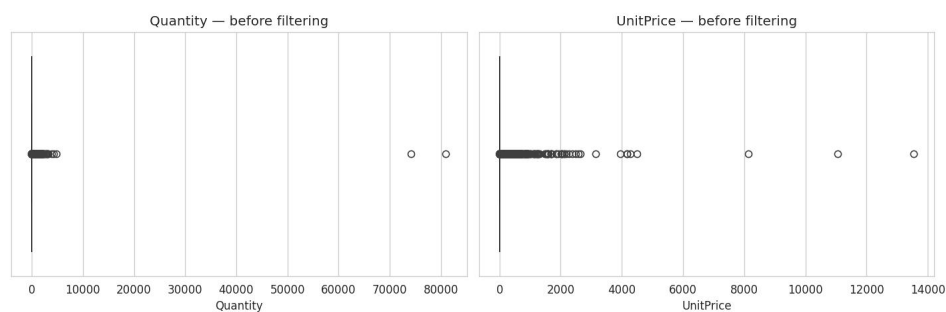


Figure 2.1 — Quantity and UnitPrice distributions before outlier filtering

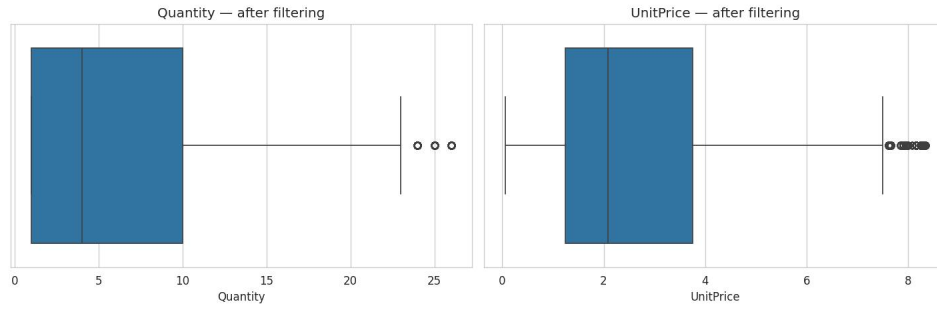


Figure 2.2 — Quantity and UnitPrice distributions after outlier filtering

## 5. Descriptive Distributions

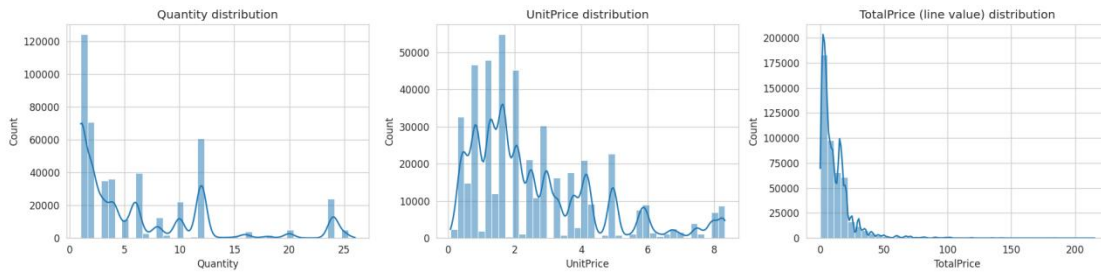


Figure 2.3 — Distributions of Quantity, UnitPrice, and TotalPrice (line value)

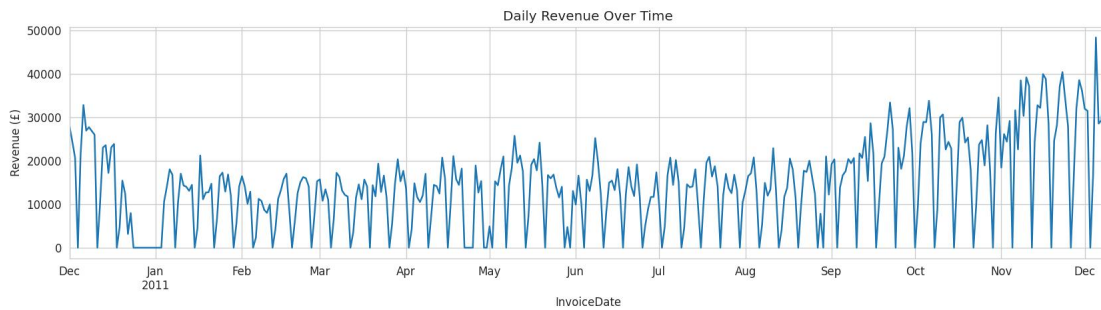


Figure 2.4 — Daily revenue over the observation window

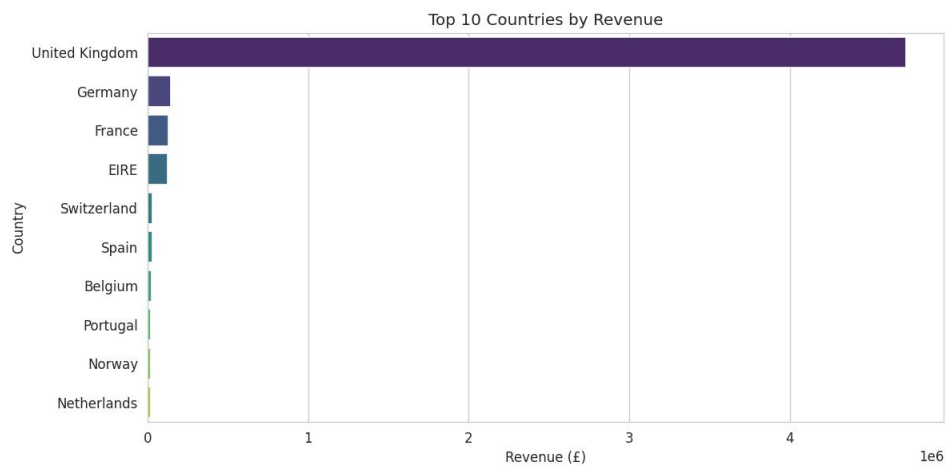


Figure 2.5 — Top 10 countries by revenue

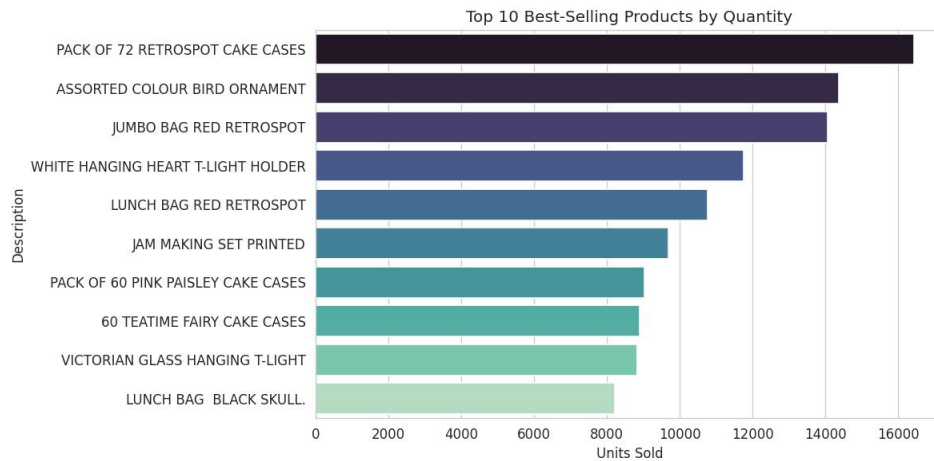


Figure 2.6 — Top 10 best-selling products by quantity

## 6. Key Patterns Identified

- Revenue trends upward toward Q4 (Oct–Dec), consistent with holiday gift-buying demand.
- Quantity and UnitPrice are both right-skewed, typical of retail transactions — most orders are small, with a long tail of bulk/wholesale orders.
- Revenue is heavily concentrated in the United Kingdom (~85%+ of transactions), with EIRE, Germany, and France as the next-largest markets.
- A small subset of SKUs accounts for a disproportionate share of units sold — a Pareto (80/20)-style demand pattern.

## Task 2 — Customer Segmentation & Behavioral Trend Dashboard

Objective: Compute RFM (Recency, Frequency, Monetary) metrics, segment customers into behavioral groups, and export clean data structures for a BI tool (Tableau / Power BI) to build interactive charts, cohort trends, and geospatial maps.

### 1. RFM Methodology

- Recency: days since each customer's most recent invoice (relative to the day after the dataset's last transaction).
- Frequency: count of distinct invoices per customer.
- Monetary: sum of TotalPrice (Quantity × UnitPrice) per customer.
- Each metric scored 1–5 by quintile (Recency inverted so recent = high score); scores summed into an RFM\_Score.

### 2. Customer Segments

Customers were classified into six behavioral segments based on their R/F score combination:

| Segment                | Customers | Avg Recency (days) | Avg Frequency | Avg Monetary (£) | Total Revenue (£) |
|------------------------|-----------|--------------------|---------------|------------------|-------------------|
| Champions              | 898       | 14.8               | 10.8          | 3,168.30         | 2,845,140.60      |
| Needs Attention        | 1,117     | 87.7               | 2.4           | 611.20           | 682,665.90        |
| Loyal Customers        | 577       | 17.4               | 3.1           | 624.40           | 360,263.90        |
| Lost / Churned         | 1,034     | 217.5              | 1.1           | 270.90           | 280,080.10        |
| At-Risk (High Value)   | 261       | 136.9              | 4.8           | 1,036.50         | 270,515.00        |
| Recent / New Customers | 309       | 18.7               | 1.2           | 324.90           | 100,382.90        |

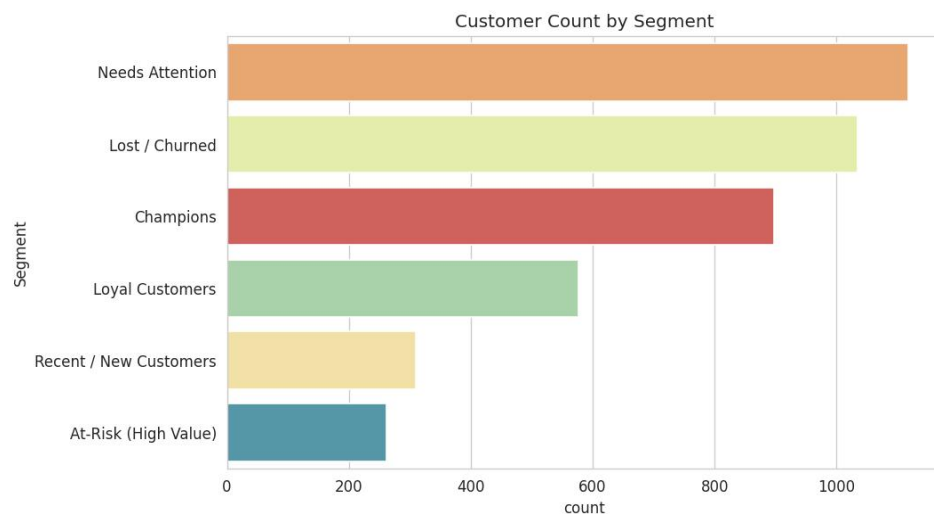


Figure 3.1 — Customer count by behavioral segment

### 3. Cohort Retention Analysis

Customers were grouped into monthly acquisition cohorts (by month of first purchase), and retention was tracked over subsequent months.

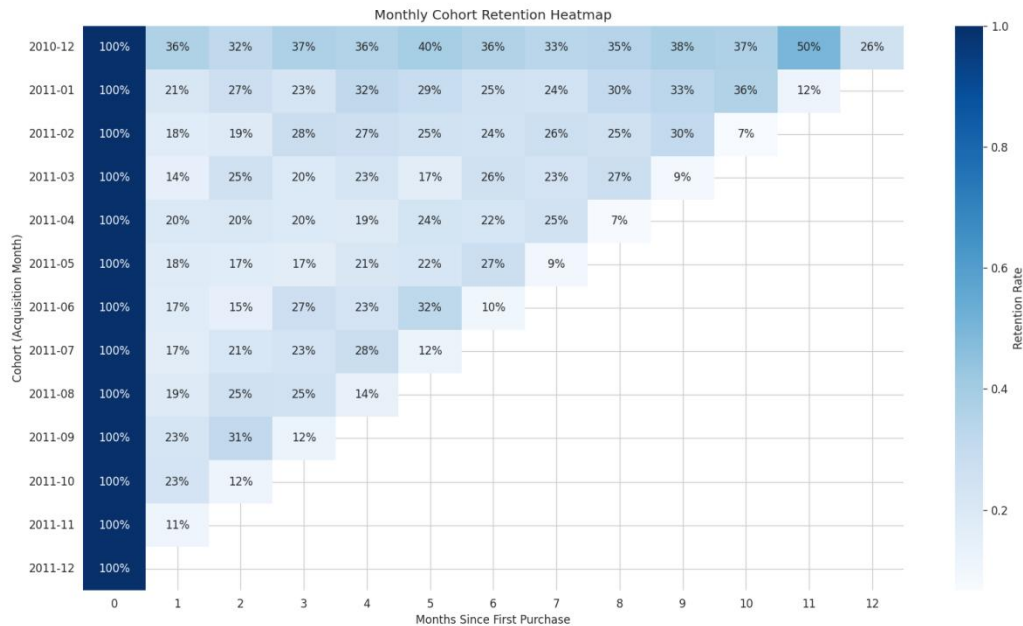


Figure 3.2 — Monthly cohort retention heatmap

## 4. Geospatial Rollup

A country-level summary (customers, orders, revenue) was exported to drive a geospatial filter map in the BI tool, allowing country selection to filter the segment and cohort views.

## 5. Exported Data Structures (BI-ready)

- customer\_rfm.csv — one row per customer: RFM metrics, R/F/M scores, RFM\_Score, Segment label.
- cohort\_retention.csv — cohort-month × month-index retention rate matrix.
- geo\_country\_summary.csv — country-level customers/orders/revenue rollup.
- segment\_summary.csv — aggregated segment-level KPIs.

## 6. Suggested Dashboard Layout (Tableau / Power BI)

- Page 1 — Segment Overview: bar chart of customers by segment, revenue treemap, KPI cards.
- Page 2 — Cohort Trends: retention heatmap, cohort size trend line.
- Page 3 — Geospatial: filled map of revenue/customers by country as a global filter.

## Task 3 — Statistical Inferential Hypothesis Analysis & Predictive Business Forecaster

Objective: Perform formal statistical significance tests on business outcomes, and build a demand forecasting model with confidence intervals.

### 1. Hypothesis Test Results

| Test           | Business Question                               | Statistic                 | p-value      | Conclusion ( $\alpha = 0.05$ )      |
|----------------|-------------------------------------------------|---------------------------|--------------|-------------------------------------|
| Welch's t-test | UK vs International avg. order value            | $t = -11.248$             | $< 0.000001$ | Reject H0 — significant difference  |
| One-way ANOVA  | Order value across top 5 countries              | $F = 22.223$              | $< 0.000001$ | Reject H0 — significant difference  |
| Chi-Square     | Market $\times$ order-size category association | $\chi^2 = 218.926$ (df=2) | $< 0.000001$ | Reject H0 — significant association |

Interpretation: All three tests reject the null hypothesis at the 5% significance level. Average order value differs significantly between UK and international customers and across the top five countries individually — supporting market-specific pricing or promotional strategy. The Chi-Square result shows order-size category is not independent of market, meaning international and UK customers differ systematically in typical order size — relevant to shipping and logistics planning.

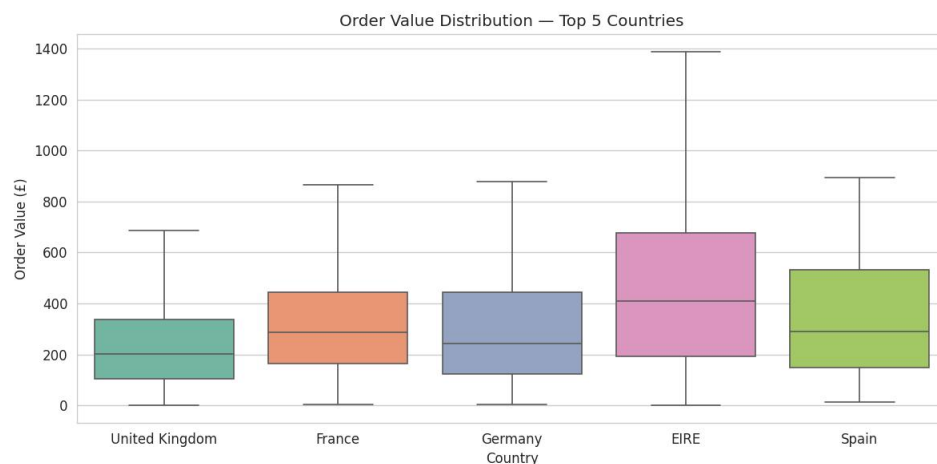


Figure 4.1 — Order value distribution across top 5 countries (ANOVA input)

### 2. Time-Series Forecasting — SARIMA

Daily revenue was modeled as a proxy for demand. Autocorrelation (ACF) and partial autocorrelation (PACF) plots, together with an Augmented Dickey-Fuller stationarity test, informed model order selection. A weekly-seasonal SARIMA(1,1,1)(1,1,1,7) model was fit to capture both trend and day-of-week seasonality.

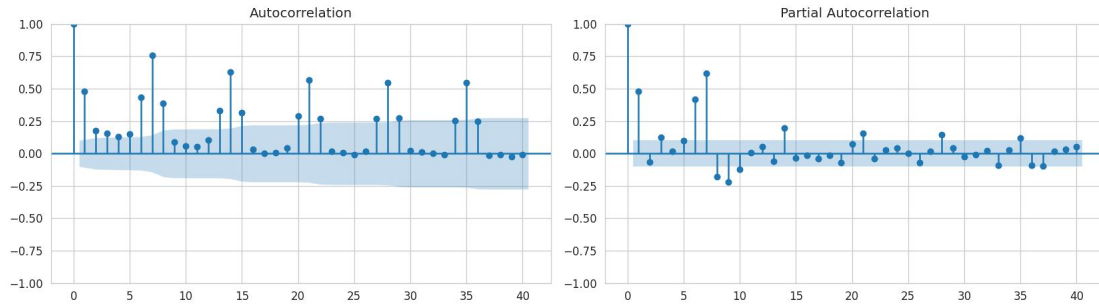


Figure 4.2 — ACF and PACF of daily revenue series

### 3. Model Validation

The model was validated on a 14-day hold-out period before being refit on the full series for forecasting, using MAE, RMSE, and MAPE error metrics (see Appendix code output).

### 4. 30-Day Demand Forecast with 95% Confidence Intervals

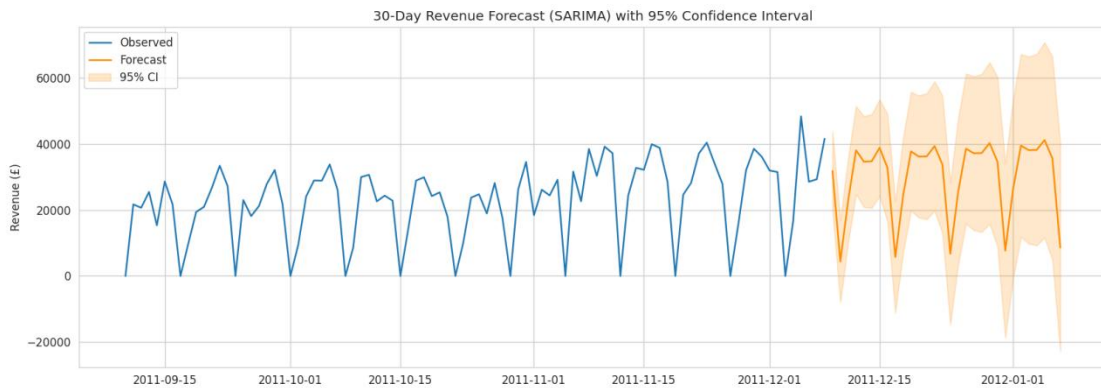


Figure 4.3 — 30-day SARIMA revenue forecast with 95% confidence band

The forecast\_results.csv file contains the full 30-day forecast table (point forecast, lower and upper 95% confidence bounds) for direct use in business planning or further BI integration.

### 5. Summary

- All three hypothesis tests found statistically significant effects, supporting differentiated strategy by market.
- The SARIMA model captures weekly seasonality in daily revenue and produces a validated 30-day forward demand projection with quantified uncertainty bounds.

## PART B — SOURCE CODE APPENDIX

The following pages contain the complete, executable Python source code for each task, in the order it was run. These are the exact code cells from the accompanying Jupyter notebooks.

### Appendix A — Task 2: EDA & Cleaning Pipeline (Source Code)

#### Code Cell 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', None)
sns.set_style('whitegrid')

df = pd.read_csv('/home/claude/data/online_retail.csv', encoding='ISO-8859-1',
                 parse_dates=['InvoiceDate'])
print("Shape:", df.shape)
df.head()
```

#### Code Cell 2

```
df.info()
```

#### Code Cell 3

```
print("Missing values per column:")
print(df.isnull().sum())
print("\nMissing % :")
print((df.isnull().mean()*100).round(2))
```

#### Code Cell 4

```
cancelled = df['InvoiceNo'].astype(str).str.startswith('C')
neg_qty = df['Quantity'] <= 0
neg_price = df['UnitPrice'] <= 0
missing_cust = df['CustomerID'].isnull()
missing_desc = df['Description'].isnull()
dupes = df.duplicated()

print(f"Cancelled invoices: {cancelled.sum():,} ({cancelled.mean()*100:.2f}%)")
print(f"Non-positive Quantity: {neg_qty.sum():,} ({neg_qty.mean()*100:.2f}%)")
print(f"Non-positive UnitPrice: {neg_price.sum():,} ({neg_price.mean()*100:.2f}%)")
print(f"Missing CustomerID: {missing_cust.sum():,} ({missing_cust.mean()*100:.2f}%)")
print(f"Missing Description: {missing_desc.sum():,} ({missing_desc.mean()*100:.2f}%)")
print(f"Exact duplicate rows: {dupes.sum():,} ({dupes.mean()*100:.2f}%)")
```

#### Code Cell 5

```
# Keep cancellations aside for return-rate analysis
returns_df = df[cancelled].copy()

# Working frame: remove cancellations
clean = df[~cancelled].copy()

# Remove non-positive quantity/price (true data errors post-cancellation-removal)
before = len(clean)
```

```

clean = clean[(clean['Quantity'] > 0) & (clean['UnitPrice'] > 0)]
print(f"Removed {before - len(clean):,} rows with non-positive Quantity/UnitPrice")

# Drop exact duplicates
before = len(clean)
clean = clean.drop_duplicates()
print(f"Removed {before - len(clean):,} duplicate rows")

# Impute missing Description via StockCode mode lookup
desc_lookup = (clean.dropna(subset=['Description'])
               .groupby('StockCode')['Description']
               .agg(lambda x: x.mode().iloc[0] if not x.mode().empty else np.nan))
missing_before = clean['Description'].isnull().sum()
clean['Description'] = clean.apply(
    lambda r: desc_lookup.get(r['StockCode'], r['Description']) if pd.isnull(r['Description']) else r['Description'],
    axis=1)
print(f"Imputed {missing_before - clean['Description'].isnull().sum():,} missing Description values via StockCode lookup")
print(f"Remaining missing Description (no lookup match): {clean['Description'].isnull().sum():,}")

clean['TotalPrice'] = clean['Quantity'] * clean['UnitPrice']
print("\nWorking dataset shape after cleaning:", clean.shape)

```

## Code Cell 6

```

def iqr_bounds(series, k=1.5):
    q1, q3 = series.quantile([0.25, 0.75])
    iqr = q3 - q1
    return q1 - k*iqr, q3 + k*iqr

fig, axes = plt.subplots(1, 2, figsize=(12,4))
sns.boxplot(x=clean['Quantity'], ax=axes[0]); axes[0].set_title('Quantity — before filtering')
sns.boxplot(x=clean['UnitPrice'], ax=axes[1]); axes[1].set_title('UnitPrice — before filtering')
plt.tight_layout()
plt.savefig('/home/claude/outputs/task2_boxplots_before.png', dpi=120)
plt.show()

```

## Code Cell 7

```

qty_low, qty_high = iqr_bounds(clean['Quantity'])
price_low, price_high = iqr_bounds(clean['UnitPrice'])
print(f"Quantity IQR bounds: [{qty_low:.2f}, {qty_high:.2f}]")
print(f"UnitPrice IQR bounds: [{price_low:.2f}, {price_high:.2f}]")

before = len(clean)
clean_filtered = clean[(clean['Quantity'].between(max(1, qty_low), qty_high)) &
                       (clean['UnitPrice'].between(max(0.01, price_low), price_high))]
print(f"\nRemoved {before - len(clean_filtered):,} outlier rows ({(before-len(clean_filtered))/before*100:.2f}%)")
print("Final cleaned shape:", clean_filtered.shape)

fig, axes = plt.subplots(1, 2, figsize=(12,4))
sns.boxplot(x=clean_filtered['Quantity'], ax=axes[0]); axes[0].set_title('Quantity — after filtering')
sns.boxplot(x=clean_filtered['UnitPrice'], ax=axes[1]); axes[1].set_title('UnitPrice — after filtering')
plt.tight_layout()
plt.savefig('/home/claude/outputs/task2_boxplots_after.png', dpi=120)
plt.show()

```

## Code Cell 8

```

fig, axes = plt.subplots(1, 3, figsize=(16,4))
sns.histplot(clean_filtered['Quantity'], bins=40, ax=axes[0], kde=True); axes[0].set_title('Quantity distribution')
sns.histplot(clean_filtered['UnitPrice'], bins=40, ax=axes[1], kde=True); axes[1].set_title('UnitPrice distribution')
sns.histplot(clean_filtered['TotalPrice'], bins=40, ax=axes[2], kde=True); axes[2].set_title('TotalPrice (line value) distribution')
plt.tight_layout()

```

```
plt.savefig('/home/claude/outputs/task2_distributions.png', dpi=120)
plt.show()
```

## Code Cell 9

```
daily_sales = clean_filtered.set_index('InvoiceDate').resample('D')['TotalPrice'].sum()
plt.figure(figsize=(14,4))
daily_sales.plot()
plt.title('Daily Revenue Over Time')
plt.ylabel('Revenue (£)')
plt.tight_layout()
plt.savefig('/home/claude/outputs/task2_daily_revenue.png', dpi=120)
plt.show()
```

## Code Cell 10

```
top_countries = clean_filtered.groupby('Country')['TotalPrice'].sum().sort_values(ascending=False).head(10)
plt.figure(figsize=(10,5))
sns.barplot(x=top_countries.values, y=top_countries.index, hue=top_countries.index, palette='viridis', legend=False)
plt.title('Top 10 Countries by Revenue')
plt.xlabel('Revenue (£)')
plt.tight_layout()
plt.savefig('/home/claude/outputs/task2_top_countries.png', dpi=120)
plt.show()
```

## Code Cell 11

```
top_products = clean_filtered.groupby('Description')['Quantity'].sum().sort_values(ascending=False).head(10)
plt.figure(figsize=(10,5))
sns.barplot(x=top_products.values, y=top_products.index, hue=top_products.index, palette='mako', legend=False)
plt.title('Top 10 Best-Selling Products by Quantity')
plt.xlabel('Units Sold')
plt.tight_layout()
plt.savefig('/home/claude/outputs/task2_top_products.png', dpi=120)
plt.show()
```

## Code Cell 12

```
clean_filtered.to_csv('/home/claude/outputs/online_retail_clean.csv', index=False)
print("Saved cleaned dataset:", clean_filtered.shape)
clean_filtered.head()
```

## Appendix B — Task 3: Customer Segmentation & Dashboard (Source Code)

### Code Cell 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style('whitegrid')
df = pd.read_csv('/home/claude/outputs/online_retail_clean.csv', parse_dates=['InvoiceDate'])

# Customer-level analysis requires a valid CustomerID
df = df.dropna(subset=['CustomerID']).copy()
df['CustomerID'] = df['CustomerID'].astype(int)
print("Shape (with valid CustomerID):", df.shape)
print("Unique customers:", df['CustomerID'].nunique())
df.head()
```

### Code Cell 2

```
snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)
print("Snapshot date (analysis 'today'):", snapshot_date)

rfm = df.groupby('CustomerID').agg(
    Recency=('InvoiceDate', lambda x: (snapshot_date - x.max()).days),
    Frequency=('InvoiceNo', 'nunique'),
    Monetary=('TotalPrice', 'sum'),
    Country=('Country', 'first')
).reset_index()

print(rfm.shape)
rfm.describe()
```

### Code Cell 3

```
# Quintile scoring (1=worst, 5=best). Recency is inverted (lower days = better = higher score)
rfm['R_score'] = pd.qcut(rfm['Recency'], 5, labels=[5,4,3,2,1]).astype(int)
rfm['F_score'] = pd.qcut(rfm['Frequency'].rank(method='first'), 5, labels=[1,2,3,4,5]).astype(int)
rfm['M_score'] = pd.qcut(rfm['Monetary'], 5, labels=[1,2,3,4,5]).astype(int)
rfm['RFM_Score'] = rfm['R_score'] + rfm['F_score'] + rfm['M_score']

def segment(row):
    if row['RFM_Score'] >= 13:
        return 'Champions'
    elif row['R_score'] >= 4 and row['F_score'] >= 3:
        return 'Loyal Customers'
    elif row['R_score'] >= 4 and row['F_score'] <= 2:
        return 'Recent / New Customers'
    elif row['R_score'] <= 2 and row['F_score'] >= 4:
        return 'At-Risk (High Value)'
    elif row['R_score'] <= 2 and row['F_score'] <= 2:
        return 'Lost / Churned'
    else:
        return 'Needs Attention'

rfm['Segment'] = rfm.apply(segment, axis=1)
rfm['Segment'].value_counts()
```

### Code Cell 4

```
plt.figure(figsize=(9,5))
```

```

order = rfm['Segment'].value_counts().index
sns.countplot(data=rfm, y='Segment', order=order, hue='Segment', palette='Spectral', legend=False)
plt.title('Customer Count by Segment')
plt.tight_layout()
plt.savefig('/home/claude/outputs/task3_segment_counts.png', dpi=120)
plt.show()

segment_summary = rfm.groupby('Segment').agg(
    Customers=('CustomerID', 'count'),
    Avg_Recency=('Recency', 'mean'),
    Avg_Frequency=('Frequency', 'mean'),
    Avg_Monetary=('Monetary', 'mean'),
    Total_Revenue=('Monetary', 'sum')
).sort_values('Total_Revenue', ascending=False).round(1)
segment_summary

```

## Code Cell 5

```

df['InvoiceMonth'] = df['InvoiceDate'].dt.to_period('M')
first_purchase = df.groupby('CustomerID')['InvoiceMonth'].min().rename('CohortMonth')
df = df.merge(first_purchase, on='CustomerID')

df['CohortIndex'] = (df['InvoiceMonth'].dt.year - df['CohortMonth'].dt.year)*12 + \
    (df['InvoiceMonth'].dt.month - df['CohortMonth'].dt.month)

cohort_data = df.groupby(['CohortMonth', 'CohortIndex'])['CustomerID'].nunique().reset_index()
cohort_pivot = cohort_data.pivot(index='CohortMonth', columns='CohortIndex', values='CustomerID')
cohort_size = cohort_pivot.iloc[:, 0]
retention = cohort_pivot.divide(cohort_size, axis=0).round(3)

plt.figure(figsize=(14, 8))
sns.heatmap(retention, annot=True, fmt='.0%', cmap='Blues', cbar_kws={'label': 'Retention Rate'})
plt.title('Monthly Cohort Retention Heatmap')
plt.ylabel('Cohort (Acquisition Month)')
plt.xlabel('Months Since First Purchase')
plt.tight_layout()
plt.savefig('/home/claude/outputs/task3_cohort_retention.png', dpi=120)
plt.show()

```

## Code Cell 6

```

geo_summary = df.groupby('Country').agg(
    Customers=('CustomerID', 'nunique'),
    Orders=('InvoiceNo', 'nunique'),
    Revenue=('TotalPrice', 'sum')
).sort_values('Revenue', ascending=False).reset_index()
geo_summary.to_csv('/home/claude/outputs/geo_country_summary.csv', index=False)
geo_summary.head(15)

```

## Code Cell 7

```

rfm.to_csv('/home/claude/outputs/customer_rfm.csv', index=False)
retention.to_csv('/home/claude/outputs/cohort_retention.csv')
segment_summary.to_csv('/home/claude/outputs/segment_summary.csv')
print("Exported: customer_rfm.csv, cohort_retention.csv, geo_country_summary.csv, segment_summary.csv")

```

## Appendix C — Task 4: Hypothesis Testing & Forecasting (Source Code)

### Code Cell 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
df = pd.read_csv('/home/claude/outputs/online_retail_clean.csv', parse_dates=['InvoiceDate'])
df.head()
```

### Code Cell 2

```
order_value = df.groupby(['InvoiceNo'])['TotalPrice'].sum().reset_index()
invoice_country = df.groupby('InvoiceNo')['Country'].first()
order_value['Country'] = order_value['InvoiceNo'].map(invoice_country)
order_value['Market'] = np.where(order_value['Country'] == 'United Kingdom', 'UK', 'International')

uk = order_value.loc[order_value['Market']=='UK', 'TotalPrice']
intl = order_value.loc[order_value['Market']=='International', 'TotalPrice']

print(f"UK orders: n={len(uk)}, mean=£{uk.mean():.2f}, std=£{uk.std():.2f}")
print(f"International orders: n={len(intl)}, mean=£{intl.mean():.2f}, std=£{intl.std():.2f}")

# Welch's t-test (unequal variance assumption is safer for real-world data)
t_stat, p_val = stats.ttest_ind(uk, intl, equal_var=False)
print(f"\nWelch's t-test: t={t_stat:.3f}, p-value={p_val:.6f}")
alpha = 0.05
if p_val < alpha:
    print(f"Result: Reject H0 (p < {alpha}) — average order value differs significantly between UK and International markets.")
else:
    print(f"Result: Fail to reject H0 (p >= {alpha}) — no significant difference detected.")
```

### Code Cell 3

```
top5_countries = df['Country'].value_counts().head(5).index.tolist()
print("Top 5 countries by transaction volume:", top5_countries)

groups = [order_value.loc[order_value['Country']==c, 'TotalPrice'] for c in top5_countries]
f_stat, p_val_anova = stats.f_oneway(*groups)
print(f"\nOne-way ANOVA: F={f_stat:.3f}, p-value={p_val_anova:.6f}")
if p_val_anova < 0.05:
    print("Result: Reject H0 — average order value differs significantly across these countries.")
else:
    print("Result: Fail to reject H0 — no significant difference across countries.")

plt.figure(figsize=(10,5))
plot_df = order_value[order_value['Country'].isin(top5_countries)]
sns.boxplot(data=plot_df, x='Country', y='TotalPrice', hue='Country', palette='Set2', legend=False, showfliers=False)
plt.title('Order Value Distribution — Top 5 Countries')
plt.ylabel('Order Value (£)')
plt.tight_layout()
plt.savefig('/home/claude/outputs/task4_anova_boxplot.png', dpi=120)
plt.show()
```

## Code Cell 4

```
order_qty = df.groupby('InvoiceNo')['Quantity'].sum().reset_index()
order_qty['Country'] = order_qty['InvoiceNo'].map(invoice_country)
order_qty['Market'] = np.where(order_qty['Country'] == 'United Kingdom', 'UK', 'International')
order_qty['SizeCategory'] = pd.qcut(order_qty['Quantity'], 3, labels=['Small','Medium','Large'])

contingency = pd.crosstab(order_qty['Market'], order_qty['SizeCategory'])
print("Contingency table:")
print(contingency)

chi2, p_val_chi, dof, expected = stats.chi2_contingency(contingency)
print(f"\nChi-Square test: chi2={chi2:.3f}, dof={dof}, p-value={p_val_chi:.6f}")
if p_val_chi < 0.05:
    print("Result: Reject H0 — order size category is significantly associated with market (UK vs International).")
else:
    print("Result: Fail to reject H0 — no significant association detected.")
```

## Code Cell 5

```
with open('/home/claude/outputs/task4_hypothesis_tests.txt', 'w') as f:
    f.write("TASK 4 - HYPOTHESIS TEST RESULTS\n")
    f.write("\n" * 50 + "\n\n")
    f.write(f"1. Welch's t-test (UK vs Intl order value): t={t_stat:.3f}, p={p_val:.6f}\n")
    f.write(f"2. One-way ANOVA (order value across top 5 countries): F={f_stat:.3f}, p={p_val_anova:.6f}\n")
    f.write(f"3. Chi-Square (market x order size category): chi2={chi2:.3f}, dof={dof}, p={p_val_chi:.6f}\n")
print("Saved hypothesis test summary.")
```

## Code Cell 6

```
daily = df.set_index('InvoiceDate').resample('D')['TotalPrice'].sum()
daily = daily[daily.index <= daily.index.max() - pd.Timedelta(days=1)] # drop partial last day
daily = daily.asfreq('D').ffill()

plt.figure(figsize=(14,4))
daily.plot()
plt.title('Daily Revenue Time Series')
plt.ylabel('Revenue (£)')
plt.tight_layout()
plt.savefig('/home/claude/outputs/task4_daily_series.png', dpi=120)
plt.show()
```

## Code Cell 7

```
fig, axes = plt.subplots(1,2, figsize=(14,4))
plot_acf(daily, lags=40, ax=axes[0])
plot_pacf(daily, lags=40, ax=axes[1])
plt.tight_layout()
plt.savefig('/home/claude/outputs/task4_acf_pacf.png', dpi=120)
plt.show()

# Augmented Dickey-Fuller stationarity test
adf_stat, adf_p, *_ = sm.tsa.adfuller(daily)
print(f"ADF statistic: {adf_stat:.3f}, p-value: {adf_p:.4f}")
print("Series is", "stationary" if adf_p < 0.05 else "non-stationary", "(at 5% level)")
```

## Code Cell 8

```
# SARIMA(1,1,1)(1,1,1,7) - weekly seasonality, first-differenced for trend/seasonal non-stationarity
train = daily.iloc[:-14]
test = daily.iloc[-14:]
```

```

model = SARIMAX(train, order=(1,1,1), seasonal_order=(1,1,1,7),
                 enforce_stationarity=False, enforce_invertibility=False)
fit = model.fit(dispatch=False)
print(fit.summary())

```

## Code Cell 9

```

# Validate on held-out last 14 days
pred = fit.get_forecast(steps=14)
pred_mean = pred.predicted_mean
conf_int = pred.conf_int(alpha=0.05)

mae = np.mean(np.abs(pred_mean.values - test.values))
rmse = np.sqrt(np.mean((pred_mean.values - test.values)**2))
mape = np.mean(np.abs((test.values - pred_mean.values) / np.where(test.values==0, np.nan, test.values))) * 100
print(f"Hold-out validation (last 14 days): MAE=£{mae:.2f}, RMSE=£{rmse:.2f}, MAPE={mape:.1f}%")

```

## Code Cell 10

```

# Refit on full series and forecast 30 days ahead
full_model = SARIMAX(daily, order=(1,1,1), seasonal_order=(1,1,1,7),
                     enforce_stationarity=False, enforce_invertibility=False)
full_fit = full_model.fit(dispatch=False)

forecast = full_fit.get_forecast(steps=30)
fc_mean = forecast.predicted_mean
fc_ci = forecast.conf_int(alpha=0.05)

plt.figure(figsize=(14,5))
plt.plot(daily.index[-90:], daily.values[-90:], label='Observed')
plt.plot(fc_mean.index, fc_mean.values, label='Forecast', color='darkorange')
plt.fill_between(fc_ci.index, fc_ci.iloc[:,0], fc_ci.iloc[:,1], color='darkorange', alpha=0.2, label='95% CI')
plt.title('30-Day Revenue Forecast (SARIMA) with 95% Confidence Interval')
plt.ylabel('Revenue (£)')
plt.legend()
plt.tight_layout()
plt.savefig('/home/claude/outputs/task4_forecast_plot.png', dpi=120)
plt.show()

forecast_table = pd.DataFrame({
    'Date': fc_mean.index,
    'Forecast': fc_mean.values.round(2),
    'Lower_95CI': fc_ci.iloc[:,0].values.round(2),
    'Upper_95CI': fc_ci.iloc[:,1].values.round(2)
})
forecast_table.to_csv('/home/claude/outputs/task4_forecast_results.csv', index=False)
forecast_table.head(10)

```